

10 — Cache Associativity e Working Sets

Pré-requisito: [04-memory-hierarchy.md](#)

Este arquivo existe porque "meu dado cabe na cache" e "eu não vou ter cache miss" são afirmações **diferentes** — e a diferença entre elas é o que explica por que tiling (blocking) funciona de verdade, não só na intuição.

1. O problema que o arquivo 04 deixou em aberto

No arquivo 04, você aprendeu que cache guarda dado recentemente usado, em blocos chamados cache lines. Mas ficou uma pergunta sem resposta: **onde exatamente**, dentro da cache, um endereço específico da RAM é guardado? Se a resposta fosse "em qualquer lugar livre", a cache precisaria checar **todas** as suas posições toda vez que você pedisse um dado — isso seria lento demais pra ser útil (lembra: cache existe justamente pra ser rápida). A solução real envolve uma restrição deliberada, que tem um efeito colateral importante.

2. Sets e ways: como a cache decide "onde" um endereço pode morar

Uma cache não é um espaço livre — ela é organizada em **sets** (conjuntos), e cada endereço de memória é mapeado, por uma conta matemática simples (geralmente alguns bits do próprio endereço), pra **um set específico**, sempre o mesmo. Dentro de cada set, existem algumas posições chamadas **ways** — o número de ways é a **associatividade** da cache.

Cache com 4 sets, associatividade 2 (2 ways por set) -- exemplo pequeno pra visualizar:

```
Set 0: [ way0 ][ way1 ]
Set 1: [ way0 ][ way1 ]
Set 2: [ way0 ][ way1 ]
Set 3: [ way0 ][ way1 ]
```

Endereço X sempre mapeia pro MESMO set (calculado a partir dos bits do endereço).
Dentro daquele set, X pode ir pra QUALQUER uma das 2 ways disponíveis.

Isso significa: a cache só precisa checar, no máximo, **associatividade** posições pra saber se um endereço já está lá — não a cache inteira. É essa restrição que mantém cache rápida. Uma L1 típica tem associatividade 8; L2 e L3 costumam ser 8-16 ways.

PERGUNTA RÁPIDA 1: se dois endereços diferentes mapeiam pro **mesmo set**, e esse set já tem todas as suas ways ocupadas por outros dados, o que acontece quando um terceiro endereço, que também mapeia pro mesmo set, precisa ser trazido pra cache?

► Resposta

3. A consequência: "cabe na cache" não é a mesma pergunta que "vai ter cache miss"

Isso é a lacuna que o arquivo 04 deixou. Imagine que seu working set (o conjunto de dados que seu código está ativamente usando, num dado momento) é menor que a L1 inteira — parece que devia caber tranquilo, sem miss. Mas se, por azar do padrão de acesso (por exemplo, strides que fazem vários endereços diferentes caírem no **mesmo set**), você está tentando guardar mais dados **naquele set específico** do que ele tem ways disponíveis, você sofre misses **mesmo com espaço de sobra em outros sets da mesma cache**.

4. Os três tipos de cache miss (taxonomia padrão da área)

- **Compulsory miss** (miss "obrigatório"): a primeira vez que você acessa um endereço, ele nunca esteve em cache antes — não tem como evitar esse, é o preço de começar a usar um dado novo.
- **Capacity miss** (miss de capacidade): seu working set é literalmente **maior** que a cache inteira — não cabe, ponto final, não importa a organização em sets.
- **Conflict miss** (miss de conflito): seu working set **caberia** na cache (capacidade suficiente), mas o padrão de acesso concentra demais em poucos sets, causando despejos desnecessários enquanto outros sets ficam ociosos — exatamente o cenário da pergunta rápida 1.

PERGUNTA RÁPIDA 2: você tem um array de 8000 floats (32 KB) e uma cache L1 de 32 KB — "cabe exatamente". Se você percorrer esse array com stride igual ao tamanho de uma linha de cache multiplicado pelo número de sets (um padrão adversarial deliberado), que tipo de miss você espera sofrer, mesmo o array "cabendo" no papel?

► Resposta

5. Working set: o conceito que junta tudo

Working set é o conjunto de dados que seu código está acessando ativamente, dentro de uma janela de tempo relevante (por exemplo, dentro de uma iteração de loop, ou dentro do corpo de uma função quente). A pergunta de otimização de cache mais importante, na prática, não é "meu array cabe na L1?" — é "**meu working set, no momento em que preciso dele todo junto, cabe (considerando organização em sets) na cache que estou mirando?**"

6. Por que isso explica tiling/blocking (a técnica que apareceu no arquivo 07)

Lembra do arquivo 07, onde o matmul com tiling tinha AI efetiva mais alta que o ingênuo? Agora você tem o mecanismo completo: tiling reorganiza o algoritmo pra processar a matriz em **blocos pequenos o suficiente pra o working set daquele bloco caber inteiro na cache** (L1 ou L2, dependendo do tamanho do tile), evitando tanto capacity miss (o bloco é deliberadamente pequeno) quanto conflict miss (o padrão de acesso dentro de um bloco pequeno tende a ser mais bem distribuído entre sets). O ganho de AI do arquivo 07 é consequência direta de reduzir esses misses — não é mágica separada, é a mesma física, com nome completo agora.

7. Por que isso importa para CUDA

Shared memory numa GPU (o equivalente aproximado de L1/scratchpad, que você vai estudar na Phase 2) também é organizada em **bancos** (banks), que se comportam de forma estruturalmente parecida com sets de cache — múltiplas threads acessando o mesmo banco simultaneamente causam **bank conflicts**, o análogo direto do conflict miss que você acabou de estudar. Tiling em CUDA (carregar um bloco de dado pra shared

memory antes de processar) é literalmente a mesma técnica do matmul com tiling em CPU, aplicada explicitamente pelo programador em vez de deixada pra cache automática.

8. Erros comuns

- Calcular só "tamanho do array vs tamanho da cache" e concluir que não vai ter miss — ignora conflict miss completamente.
- Achar que mais associatividade sempre ajuda infinitamente — existe um trade-off real (mais ways custa mais silício, mais energia, checagem um pouco mais lenta); a associatividade real de qualquer CPU é um número fixo, escolhido pelo fabricante.
- Confundir working set com "todo o dado que o programa usa alguma vez" — working set é sobre o que está **ativamente em uso numa janela de tempo**, não o dataset inteiro.

PREDICT BEFORE RUNNING (conceitual)

Uma cache L1 de 32 KB, associatividade 8, linha de 64 bytes (então tem 64 sets: $32\text{KB} / 64\text{B linha} / 8 \text{ ways} = 64$ sets).

1. Você acessa 8 arrays diferentes, cada um pequeno, mas todos com o **mesmo** endereço base módulo o tamanho de um set (um padrão adversarial artificial). O que acontece quando você tenta acessar um nono array com esse mesmo padrão?
2. Isso seria capacity miss ou conflict miss?
3. Se você reorganizasse esses 9 arrays com um pequeno deslocamento entre eles (padding), quebrando o alinhamento adversarial, o que mudaria?

► Explicação